

UNIT I:

- **Introduction:** Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- **Entity Relationship Model:** Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

UNIT I: Introduction

- **Database system**, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Database system

- **Data:**

- Collection of ***raw facts*** having no particular meaning (e.g. 1, ABC, 19 etc). Or
- Data is a collection of a distinct small unit of information. It can be used in a variety of forms like text, numbers, media, bytes, etc.

- **Record:** Collection of ***related data items***, e.g. in the above example the three data items had no meaning.

Database system

- But if we organize them in the following way, then they collectively represent meaningful information

Roll	Name	Age
1	ABC	19

- **Table or Relation:** Collection of related records.

Roll	Name	Age
1	ABC	19
2	DEF	19
3	XYZ	20

Database system

- The **columns** of this relation are called Fields, Attributes or Domains.
- The **rows** are called Tuples or Records.
- **Database:** Collection of related relations. Consider the following collection of tables:

T1

	Roll	Name	Age
1		ABC	19
2		DEF	22
3		XYZ	28

T2

	Roll	Address
1		KOL
2		DEL
3		MUM

Database system

- A database is an **organized collection of structured information, or data**, typically stored electronically in a computer system.
- A database is usually controlled by a **database management system (DBMS)**.
- A Database Management System (**DBMS**) is a software system that is designed to **manage** and **organize** data in a structured manner. It **allows users to create, modify, and query** a database, as well as manage the security and access controls for that database.

UNIT I: Introduction

- Database system, **Characteristics (Database Vs File System)**, Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Characteristics (Database Vs File System)

- The Difference between file system and dbms(database management system) is that a file system aids in the **storage of a collection of raw data files on a hard disc.**
- In contrast, a database management system (DBMS) is a software system that aids in storing, manipulating, and recovering data.

Characteristics (Database Vs File System)

DBMS	File System
Dbms is a collection of data or software to store and retrieve the user's data.	File system is used to manage and organize the files stored in the computer's hard disk.
Dbms gives an abstract view of data that hides the details	The file system specifies the details of data representation and storage.
Dbms provide a backup means data lost can be recovered	In the file system lost data cannot be recovered.
The database system is expensive and complex to design	The file system is cheaper and simple to design

Characteristics (Database Vs File System)

DBMS	File System
No Data Redundancy in DBMS	Data Redundancy will be high in file system
Dbms provide multiple users interfaces	Data is isolated in the file system
In DBMS, efficient query processing is available.	In the file system, there is no efficient query processing .
In Dbms, there is more data consistency because of the process of normalization.	The file system's data consistency is poor .
Example: Oracle, MySQL, MS SQL Server, DB2, Microsoft Access, and other databases	Example: Cobal, .doc, .txt, .png

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), **Database Users**, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Database Users

- Database Administrators (DBA)
- Database Designers
- System Analysts
- Application Programmers / Back-End Developers
- Naive Users / Parametric Users
- Sophisticated Users
- Casual Users / Temporary Users

Database Users

- Database Administrators (DBA)
- Database Designers
- System Analysts
- Application Programmers / Back-End Developers
- Naive Users / Parametric Users
- Sophisticated Users
- Casual Users / Temporary Users

Database Users

- **Design of the Conceptual and Physical Schemas:** The DBA is responsible for interacting with the users of the system to understand what data is to be stored in the DBMS and how it is likely to be used.
- **Security and Authorization:** The DBA is responsible for ensuring that unauthorized data access is not permitted. In general, not everyone should be able to access all the data. In a relational DBMS, users can be granted permission to access only certain views and relations.

Database Users

- **Data Availability and Recovery from Failures:** The DBA must take steps to ensure that if the system fails, users can continue to access as much of the uncorrupted data as possible. The DBA must also work to restore the data to a consistent state.
- **Database Tuning:** Users' needs are likely to evolve with time. The DBA is responsible for modifying the database, in particular the conceptual and physical schemas, to ensure adequate performance as requirements change.

Database Users

- Database Administrators (DBA)
- Database Designers
- System Analysts
- Application Programmers / Back-End Developers
- Naive Users / Parametric Users
- Sophisticated Users
- Casual Users / Temporary Users

Database Users

- **Database Designers:** As the name suggests, Database Designers are the users in DBMS who design and create the structure of the database including triggers, indexes, schemas, entity relationships, tables, constraints, etc. which complete the database.
- **Parametric End Users:** These are the unsophisticated users who don't have any DBMS knowledge but they frequently use the database applications in their daily life to get the desired results. For example, Clerks in any bank is a naive user because they don't have any DBMS knowledge but they still use the database and perform their given task.

Database Users

- **A System Analyst:** A system Analyst is a user **who analyzes the requirements of parametric end users**. They check whether all the requirements of end users are satisfied.
- **Sophisticated Users:** Sophisticated users can be **engineers, scientists, business analyst, who are familiar with the database**. They can develop their own database applications according to their requirement. **They don't write the program code but they interact the database by writing SQL queries** directly through the query processor.

Database Users

- **Application Programmers** also known as **Back-End Developers**, are computer professional **users who are responsible for developing the application programs** (C, C++, Java, PHP, Python, etc.) or the user interface so that other users can use these applications to interact with the database.
- **Casual Users / Temporary Users:** Casual Users are the users who **occasionally use/access the database** but each time when they access the database they require the new information, for example, Middle or higher level manager.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, **Advantages of Database systems**, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Advantages of Database systems

- **Data Independence:** The DBMS provides an **abstract view** of the data that hides such details.
- **Efficient Data Access:** A DBMS utilizes a variety of sophisticated **techniques to store and retrieve** data efficiently.
- **Data Integrity and Security:** If data is always accessed through the DBMS, the DBMS can enforce **integrity constraints**. For example it can enforce access controls that govern **what data is visible to different classes of users**.

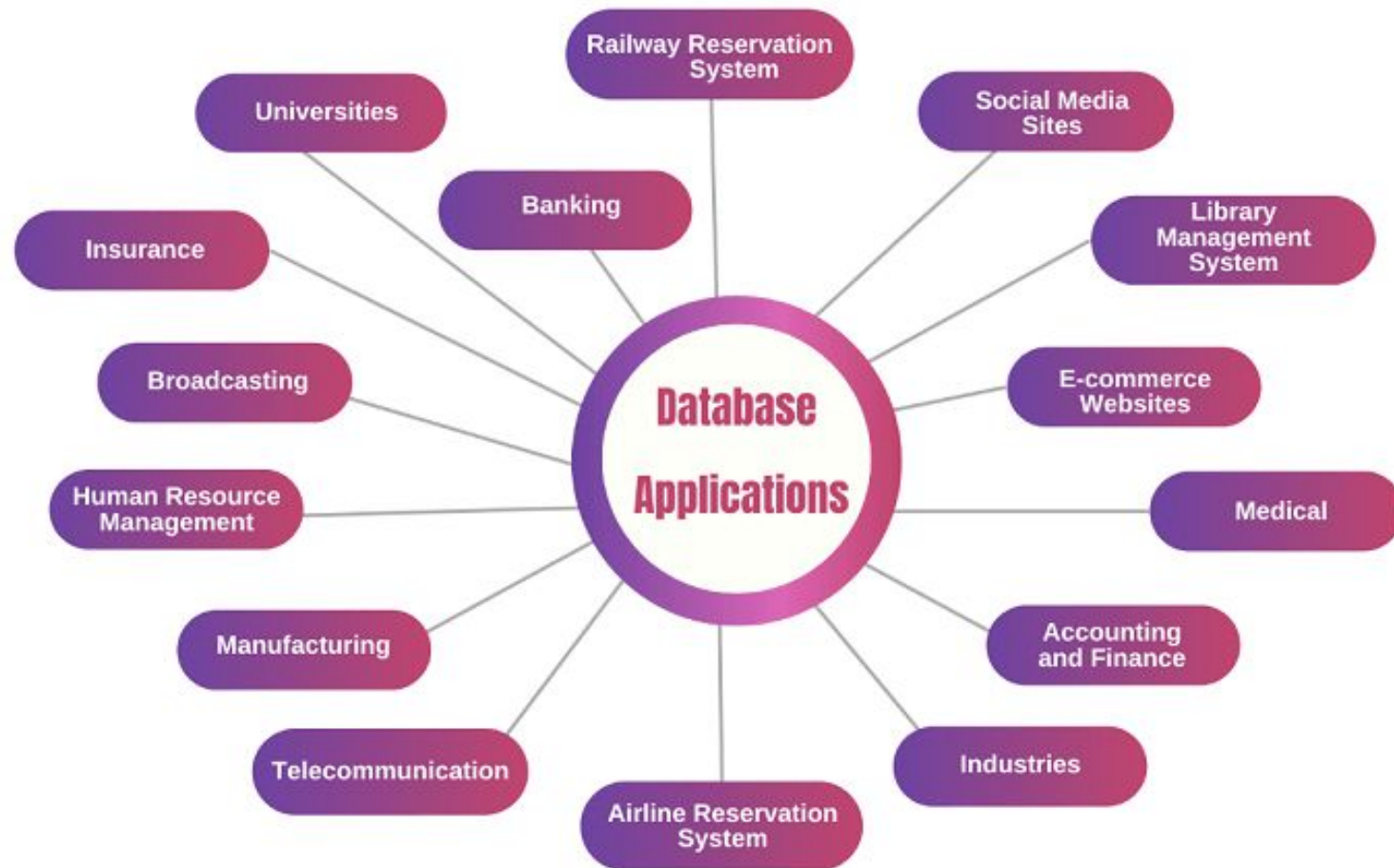
Advantages of Database systems

- **Data Administration:** Experienced professionals who understand the nature of the data being managed, and how different groups of users use it, can be responsible for organizing the data representation to **minimize redundancy**.
- **Concurrent Access and Crash Recovery:** A DBMS schedules concurrent accesses to the data in such a manner that users can think of the **data is being accessed by only one user at a time**.
- **Reduced Application Development Time:** Clearly, the DBMS **supports important functions** that are common to many applications accessing data in the DBMS.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, **Database applications**. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Database applications



UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. **Brief introduction of different Data Models**; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - **Relational Data Model**
 - **Network Data Model**
 - **Hierarchical Data Model**
 - **Object based Data Models.**
 - **Object oriented model**
 - **E - R Model**

Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - **Relational Data Model**
 - **Network Data Model**
 - **Hierarchical Data Model**
 - **Object based Data Models.**
 - **Object oriented model**
 - **E - R Model**

Data Models: Relational Model

- The relational model **uses tables** to represent the data and the relationships among those data.
- Each table has **multiple columns**, and each column is identified by a unique name.
- In this database, each **row** in the table represents a different employee.
- Relationships link rows from two tables on the basis of the key field.

Data Models: Relational Model

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

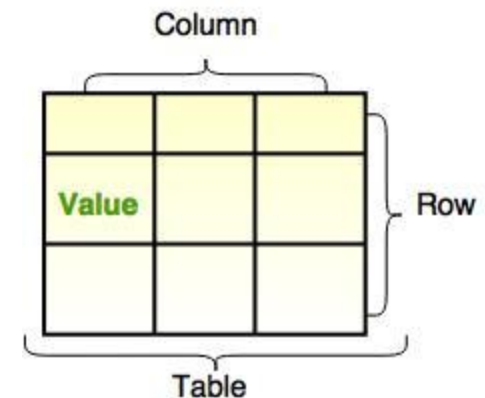


Fig. Relational Model

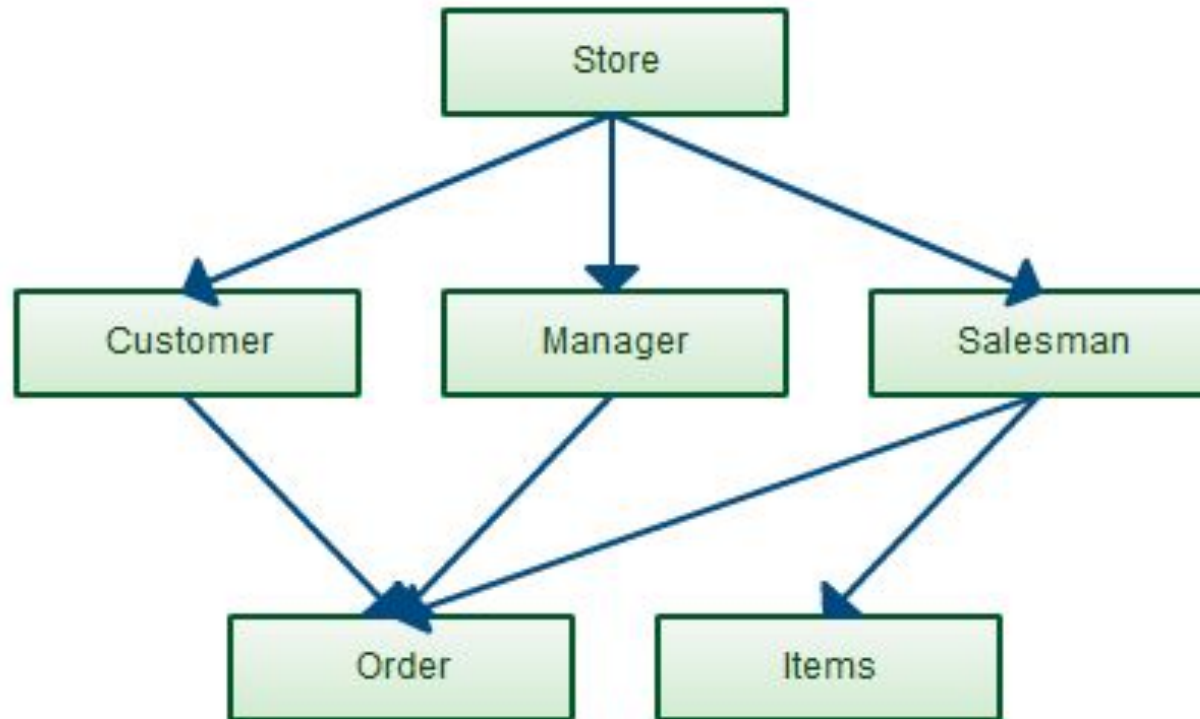
Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - Relational Data Model
 - **Network Data Model**
 - Hierarchical Data Model
 - **Object based Data Models.**
 - Object oriented model
 - E - R Model

Data Models: Network Data Model

- In the network model, data are represented by collections of records.
- Relationships among data are represented by links.
- In this model Graph data structure is used.
- A network model permits a record to have more than one parent.

Data Models: Network Data Model



Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - Relational Data Model
 - Network Data Model
 - **Hierarchical Data Model**
 - **Object based Data Models.**
 - Object oriented model
 - E - R Model

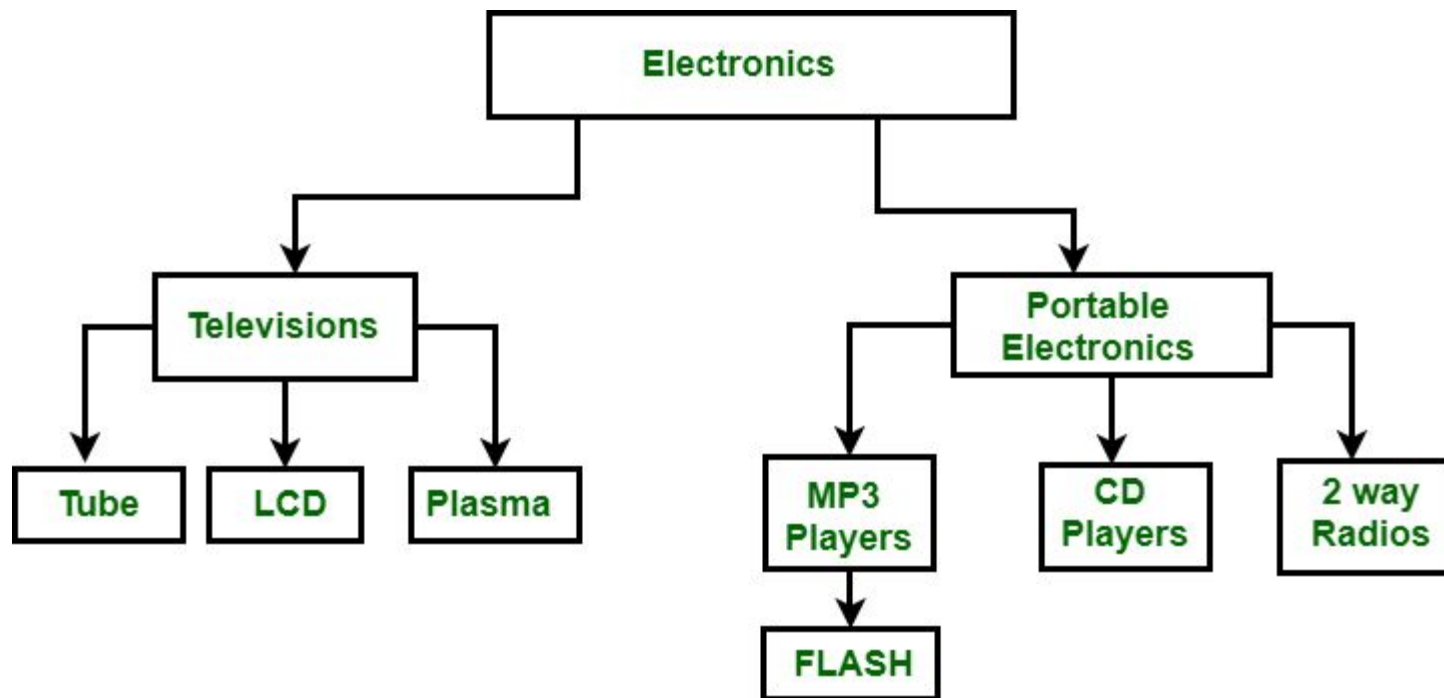
Data Models: Hierarchical Model

- A hierarchical database is a data model in which data is stored in the form of records.
- A hierarchical database organized into a **tree-like structure**, or parent-child structure in which one **parent node can have many child nodes** connected through links.
- The hierarchical database structure dictates that, while a parent record can have several child records, **each child record can only have one parent record**.

Data Models: Hierarchical Model

- Retrieving hierarchical data from a hierarchical database architecture requires traversing the entire tree, starting at the root node.

Data Models: Hierarchical Model



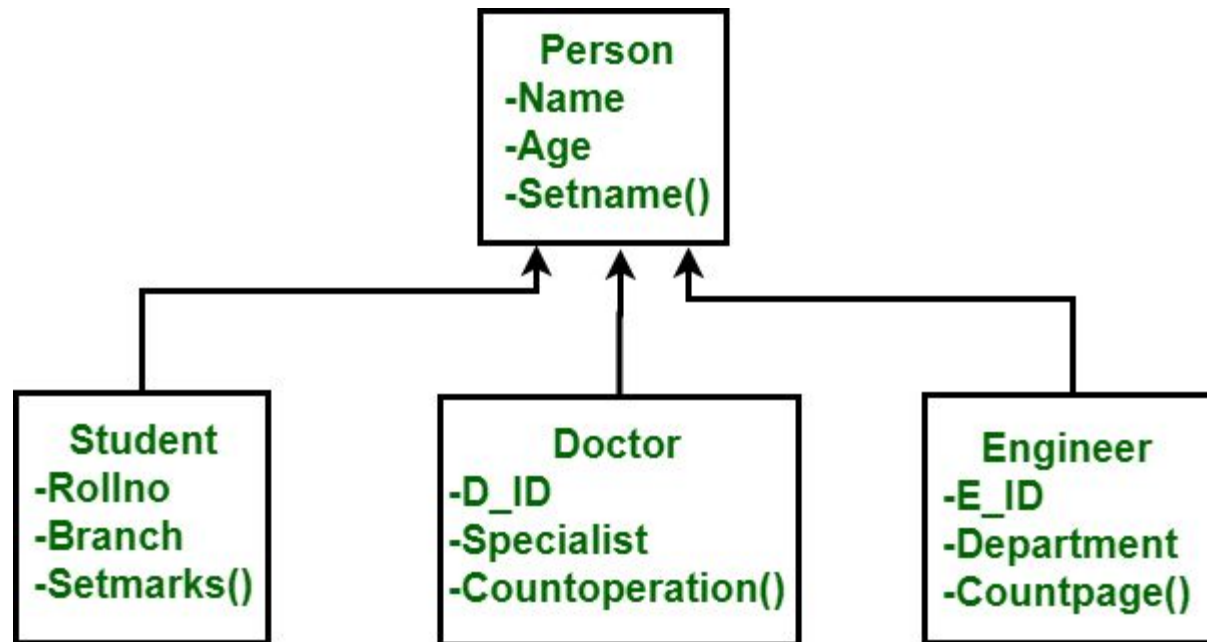
Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - Relational Data Model
 - Network Data Model
 - Hierarchical Data Model
 - **Object based Data Models.**
 - **Object oriented model**
 - E - R Model

Data Models:Object Oriented Model

- The **real-world problems** are closely represented through the object-oriented data model.
- In this type of model, both the data and relationship are represented in a single structure called an **object**.
- In this model, the **attributes describe the properties** of an object.
- Objects that share similar characteristics are grouped in **classes**.
- Therefore, a **class is a collection of similar objects** with attributes and methods.

Data Models:Object Oriented Model



Data Models

- There are basically two types of data models
 - **Record based Data Models.**
 - Relational Data Model
 - Network Data Model
 - Hierarchical Data Model
 - **Object based Data Models.**
 - Object oriented model
 - **E - R Model**

Data Models: E-R Model

- ❑ An entity is **an object in the real world** that is distinguishable from other objects.
- ❑ This is the **basic object of E-R Model**.
- ❑ Entities can be represented by '**Rectangle**'.
- ❑ Example: A diagram illustrating the representation of an entity. It consists of a light blue rectangle, a light blue arrow pointing to the right, and the word "Entity" in a small, light blue font.
- ❑ In a school database students, teachers, classes, and courses offered can be considered as entities.

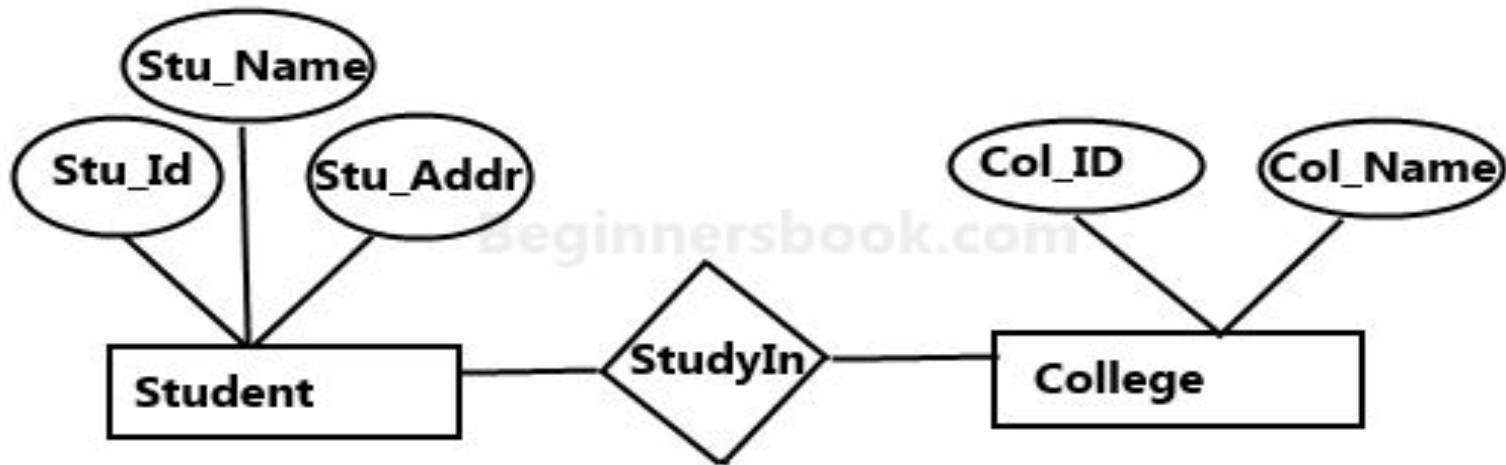
Data Models: E-R Model

- Entities have *attributes*
 - **Example:** people have *names* and *addresses*
- An **entity set** is a set of entities of the same type that share the same properties.
 - **Example:** set of all persons, companies, trees, holidays

Roll No.	Name	Course	
CS08	Steive	Comp. Sci.	→ Entity
EE54	Jhoson	Electronics	→ Entity
B12	Eva	Biology	→ Entity
F32	Jhoson	Finance	→ Entity
M26	Erica	Maths	→ Entity

Student Table

Data Models: E-R Model



Sample E-R Diagram

Data Models: E-R Model

- ❑ Entities are represented by means of their properties, called **attributes**.
- ❑ For example, a student entity may have name, class, and age as attributes.
- ❑ **Domain** – the set of permitted values for each attribute.
- ❑ For example,
 - ❑ A student's name cannot be a numeric value.
 - ❑ A student's age cannot be negative, etc.

Data Models: E-R Model

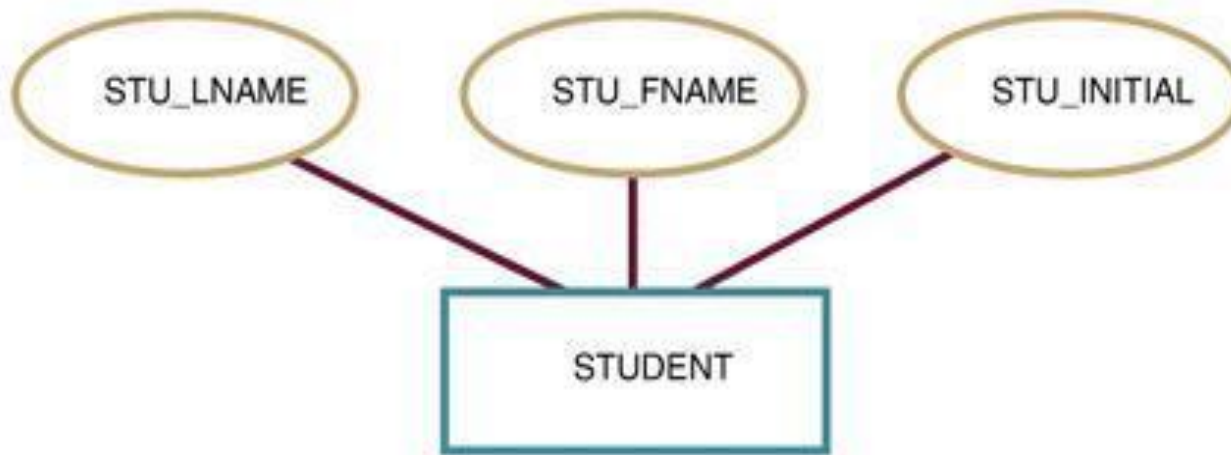


FIGURE 4.6 ■ THE ATTRIBUTES OF THE STUDENT ENTITY

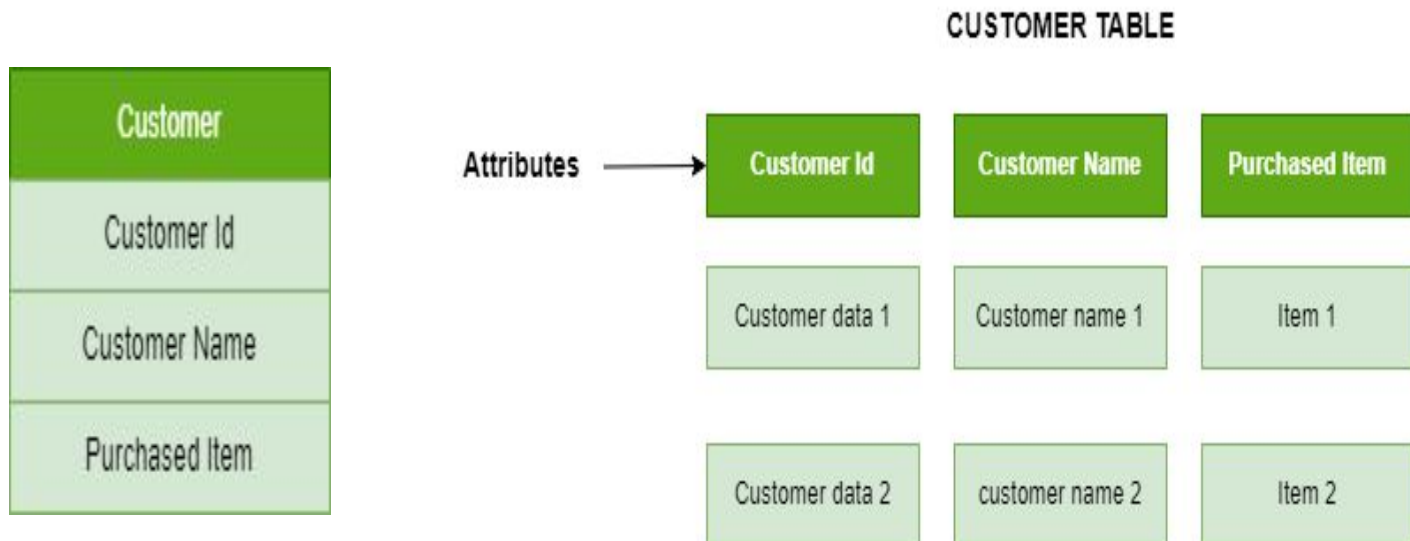
UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; **Concepts of Schema, Instance and data independence**; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Concepts of Schema, Instance and data independence

- The overall design of the database is called Schema.
- A database schema is a logical representation of data that shows how the data in a database should be stored logically.
- It shows how the data is organized and the relationship between the tables.
- Database schema defines a set of guidelines that control the database along with that it provides information about the way of accessing and modifying the data.

Concepts of Schema, Instance and data independence

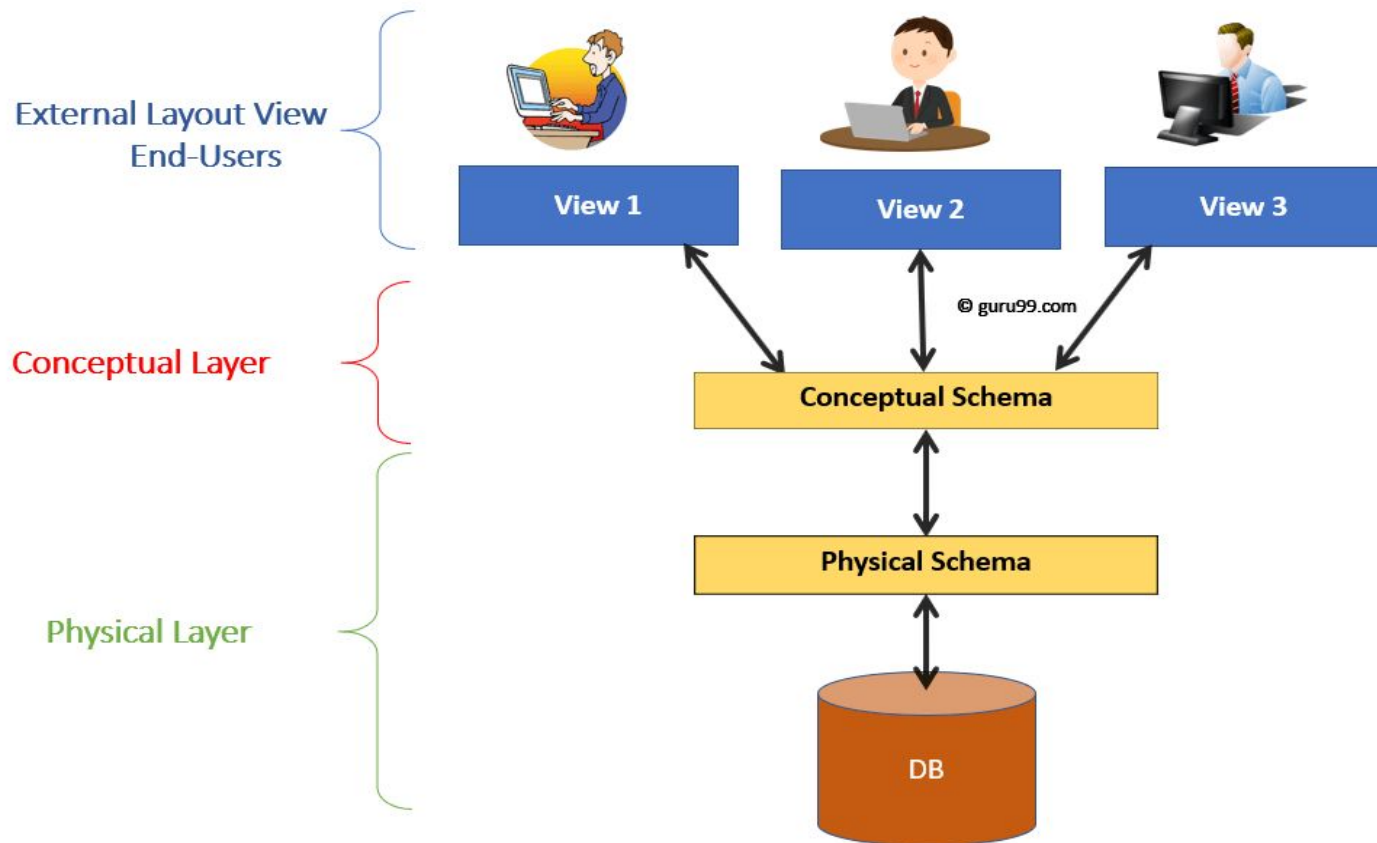


Concepts of Schema, Instance and data independence

- The database description consists of a schema at each of these three levels of abstraction: **the conceptual, physical, and external.**
- A data definition language (**DDL**) is used to define the external and conceptual schemas.
- The conceptual schema (**sometimes called the logical schema**) describes the stored data in terms of the data model of the DBMS.

Concepts of Schema, Instance and data independence

□ Three Level Architecture of DBMS:



Concepts of Schema, Instance and data independence: External Level

- It is also called **view level**.
- The reason this level is called “view” is because **several users can view** their desired data from this level.
- The **user doesn't need to know** the database schema details such as data structure, table definition etc.
- User is **only concerned about data** which is what returned back to the view level
- External level is the “**top level**” of the Three Level DBMS Architecture.

Concepts of Schema, Instance and data independence: Conceptual Level

- It is also known as **logical level**.
- The conceptual level is a higher level than the physical level.
- This level is maintained by **DBA(database administrator)**.
- The whole design of the database such as **relationship among data ,schema of data** etc.. are described in this level.
- It is the **middle level** between external or internal levels.

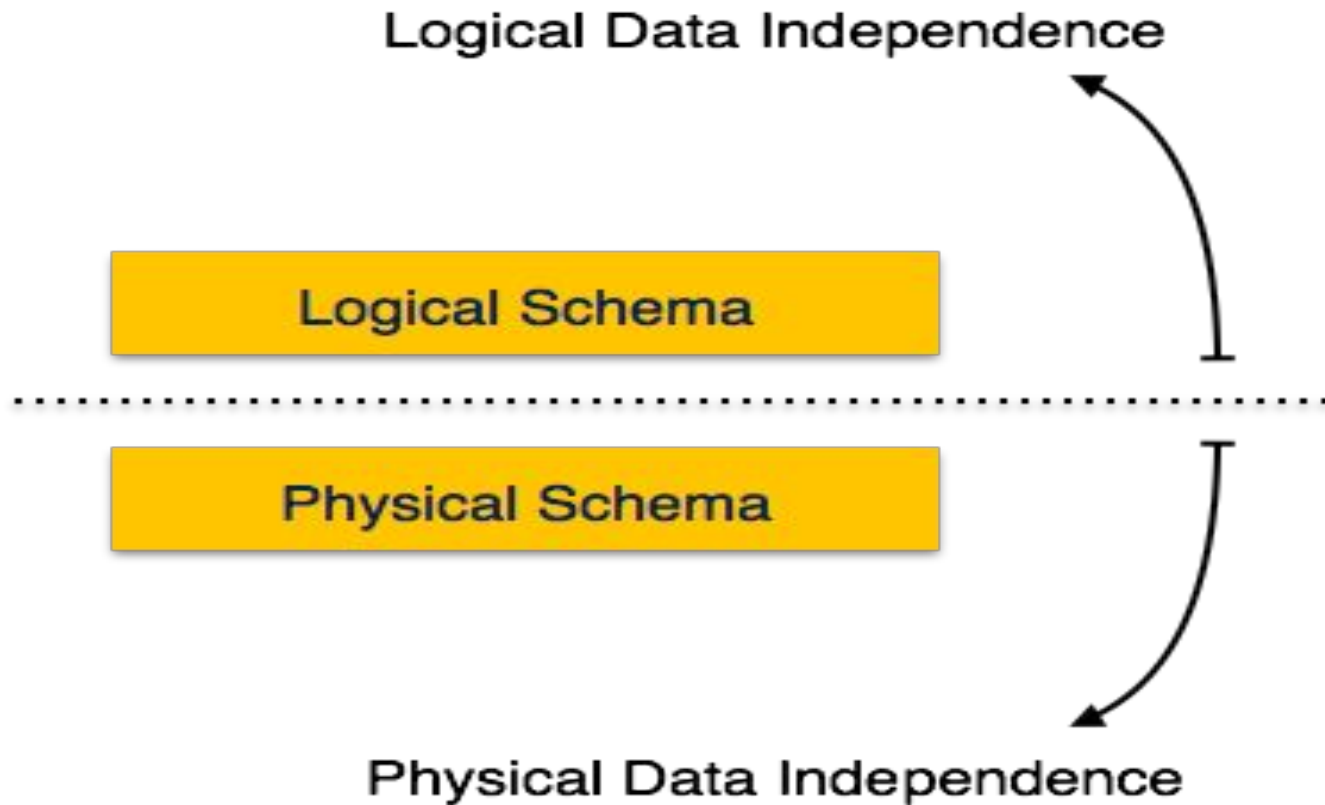
Concepts of Schema, Instance and data independence: Internal Level

- It is also known as **physical level**.
- This is the **lowest level** of the architecture.
- This data is **stored in the external hard drives** in the form of bits.
- This level describes **how the data actually stored** in the storage device.
- This level is also responsible for **allocating space** of the data.

Concepts of Schema, Instance and data independence

- The three-schema architecture can be used to further explain the concept of data independence.
- **Data independence** can be defined as the **capacity to change the schema** at one level of a database system without having to change the schema at the next higher level.
- We can define two types of data independence:
 - **Physical data independence**
 - **Logical data independence**

Concepts of Schema, Instance and data independence



Concepts of Schema, Instance and data independence

□ **Physical data independence**

- Is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.

□ **Logical data independence**

- Is the capacity to change the conceptual schema without having to change external schema or application programs.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; **Three tier schema architecture for data independence**; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Three tier schema architecture for data independence



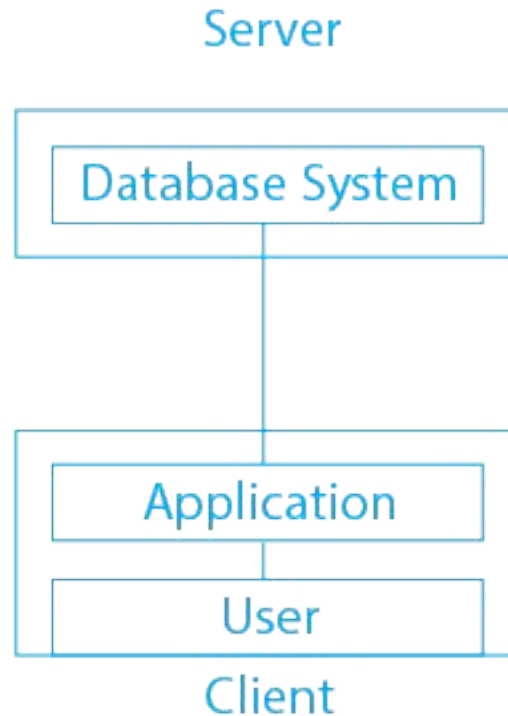
- 2-tier Architecture
- 3-tier Architecture

Three tier schema architecture for data independence



- 2-tier Architecture
- 3-tier Architecture

Three tier schema architecture for data independence



In a 2- tier architecture, client connects directly to Database

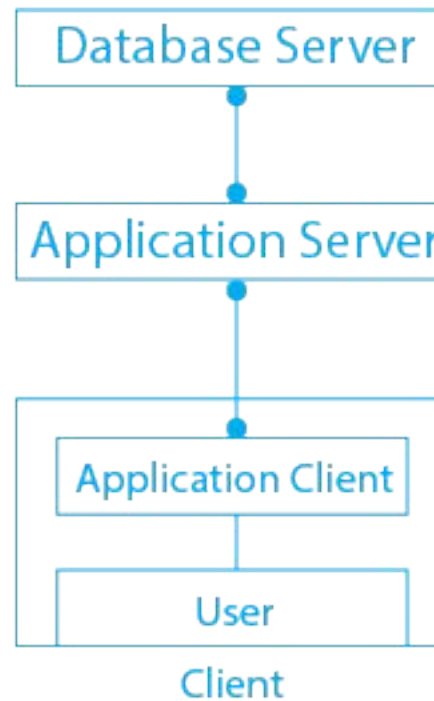
Three tier schema architecture for data independence

- 2 tier architecture in DBMS refers to a **client-server architecture** where the **user interface** and the **application logic** are separated into two separate components.
- The **client component** is typically the user interface and the **server component** is responsible for handling the data and business logic.
- In this architecture, the client component communicates directly with the server component to request data and perform actions.

Three tier schema architecture for data independence

- 2-tier Architecture
- 3-tier Architecture

Three tier schema architecture for data independence



3- tier Architecture

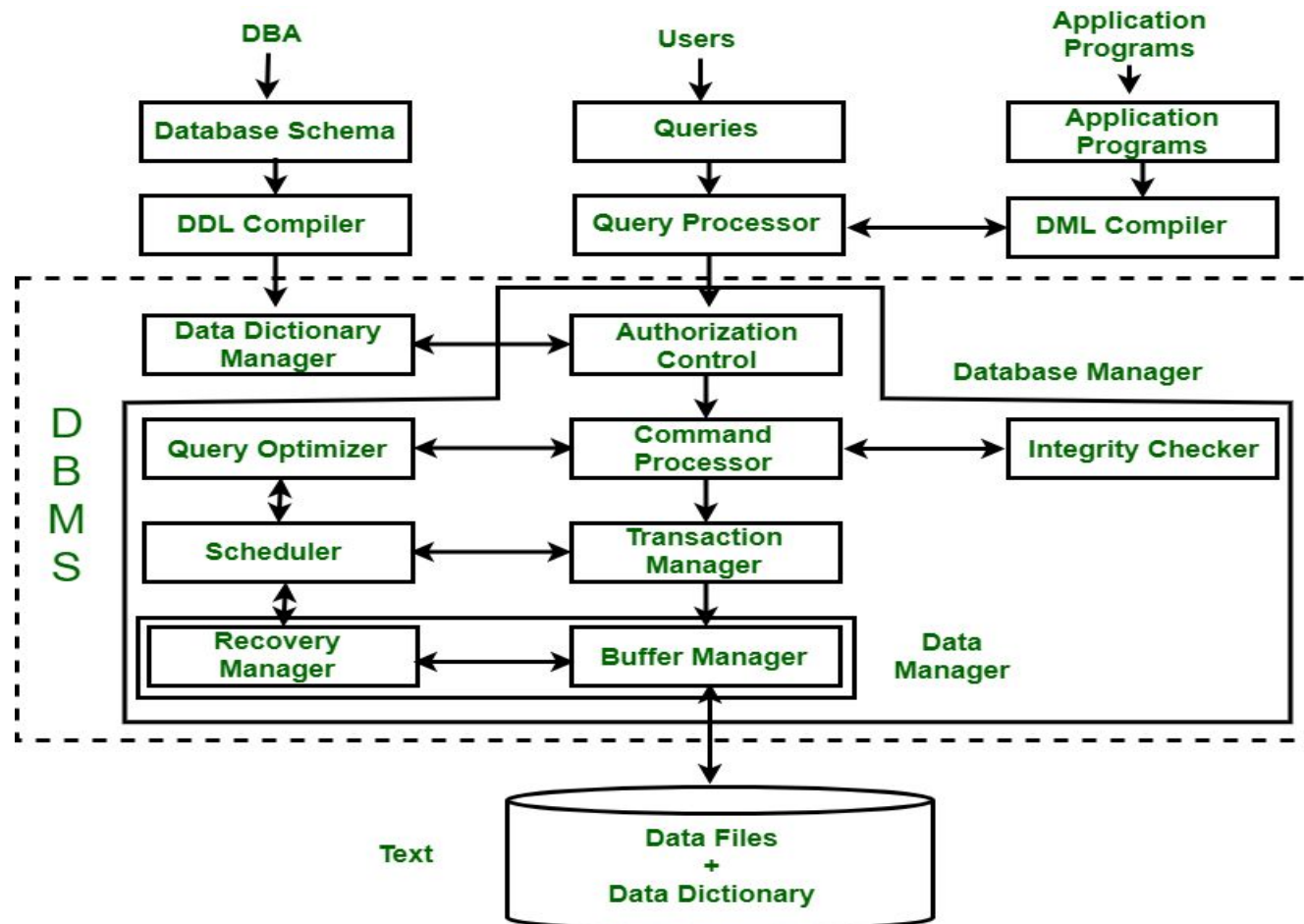
Three tier schema architecture for data independence

- The **client cannot communicate directly with the server** with this design.
- On the client side, the **program communicates with an application server**, which then communicates with the database system.
- Beyond the application server, the **end-user has no knowledge of the database's existence**.
- Aside from the application, the **database has no knowledge of any other users**.
- In the event of an extensive **web application**, the 3-tier design is used.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; **Database system structure, environment**, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Database system structure, environment



Database system structure, environment

- The database system is divided into three components:
 - Query Processor
 - Storage Manager
 - Disk Storage.

Database system structure, environment

- The database system is divided into three components:
 - Query Processor
 - Storage Manager
 - Disk Storage.

Database system structure, environment

- **Query Processor:** It **interprets the requests** (queries) received from end user via an application program into instructions. It also executes the user request which is received from the **DML compiler**. Query Processor contains the following components:
 - **DML Compiler:** It processes the DML statements into low level instruction (machine language), so that they can be executed.
 - **DDL Interpreter:** It processes the DDL statements into a set of table containing meta data (data about data).
 - **Embedded DML Pre-compiler:** It processes DML statements embedded in an application program into procedural calls.
 - **Query Optimizer:** It executes the instruction generated by DML Compiler.

Database system structure, environment

- The database system is divided into three components:
 - Query Processor
 - **Storage Manager**
 - Disk Storage.

Database system structure, environment

- **Storage Manager:**
- Storage Manager is a program that provides an interface between the data stored in the database and the queries received.
- It is also known as Database Control System.
- It is responsible for updating, storing, deleting, and retrieving data in the database. It contains the following components –
- **Authorization Manager:** It ensures role-based access control, i.e., checks whether the particular person is privileged to perform the requested operation or not.

Database system structure, environment

- **Integrity Manager:** It checks the integrity constraints when the database is modified.
- **Transaction Manager:** It controls concurrent access by performing the operations in a scheduled way that it receives the transaction.
- **File Manager:** It manages the file space and the data structure used to represent information in the database.
- **Buffer Manager:** It is responsible for cache memory and the transfer of data between the secondary storage and main memory.

Database system structure, environment

- The database system is divided into three components:
 - Query Processor
 - Storage Manager
 - Disk Storage.

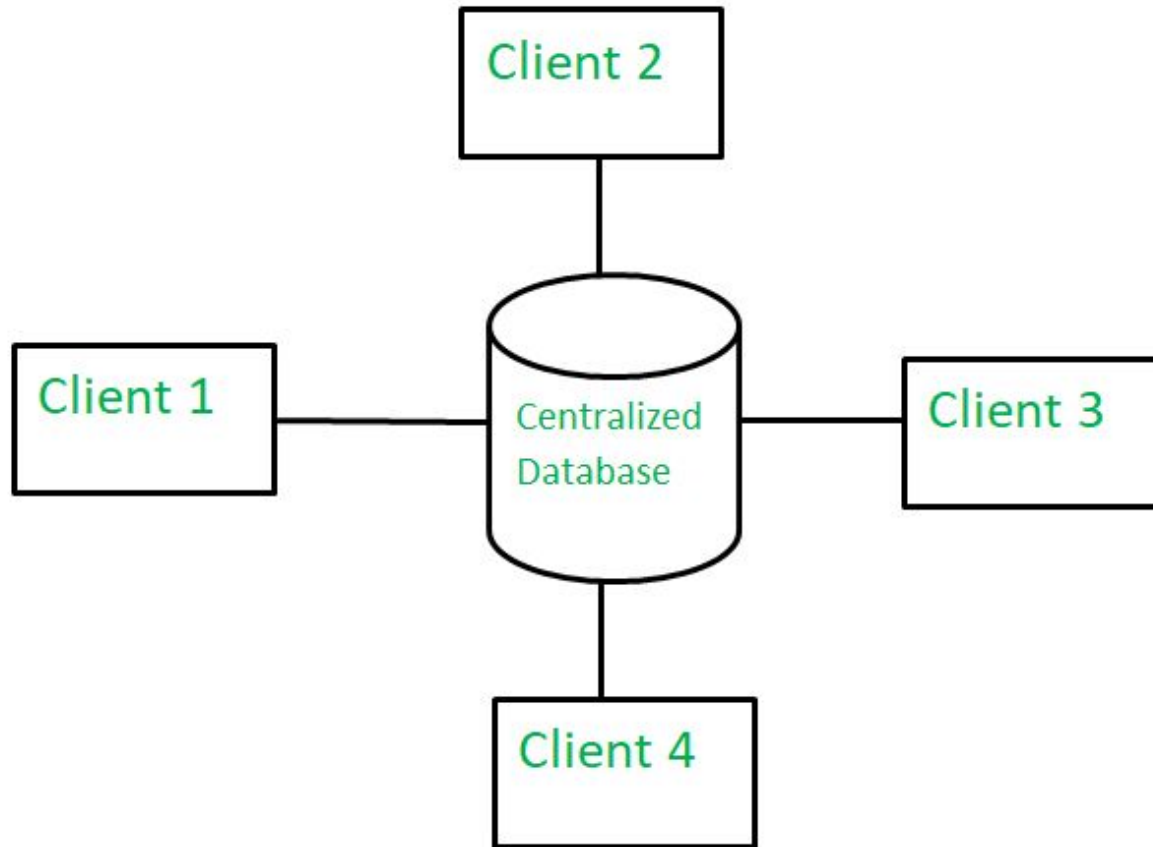
Database system structure, environment

- **Disk Storage:** It contains the following components:
- **Data Files:** It stores the data.
- **Data Dictionary:** It contains the information about the structure of any database object.
- **Indices:** It provides faster retrieval of data item.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, **Centralized and Client Server architecture for the database.**
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

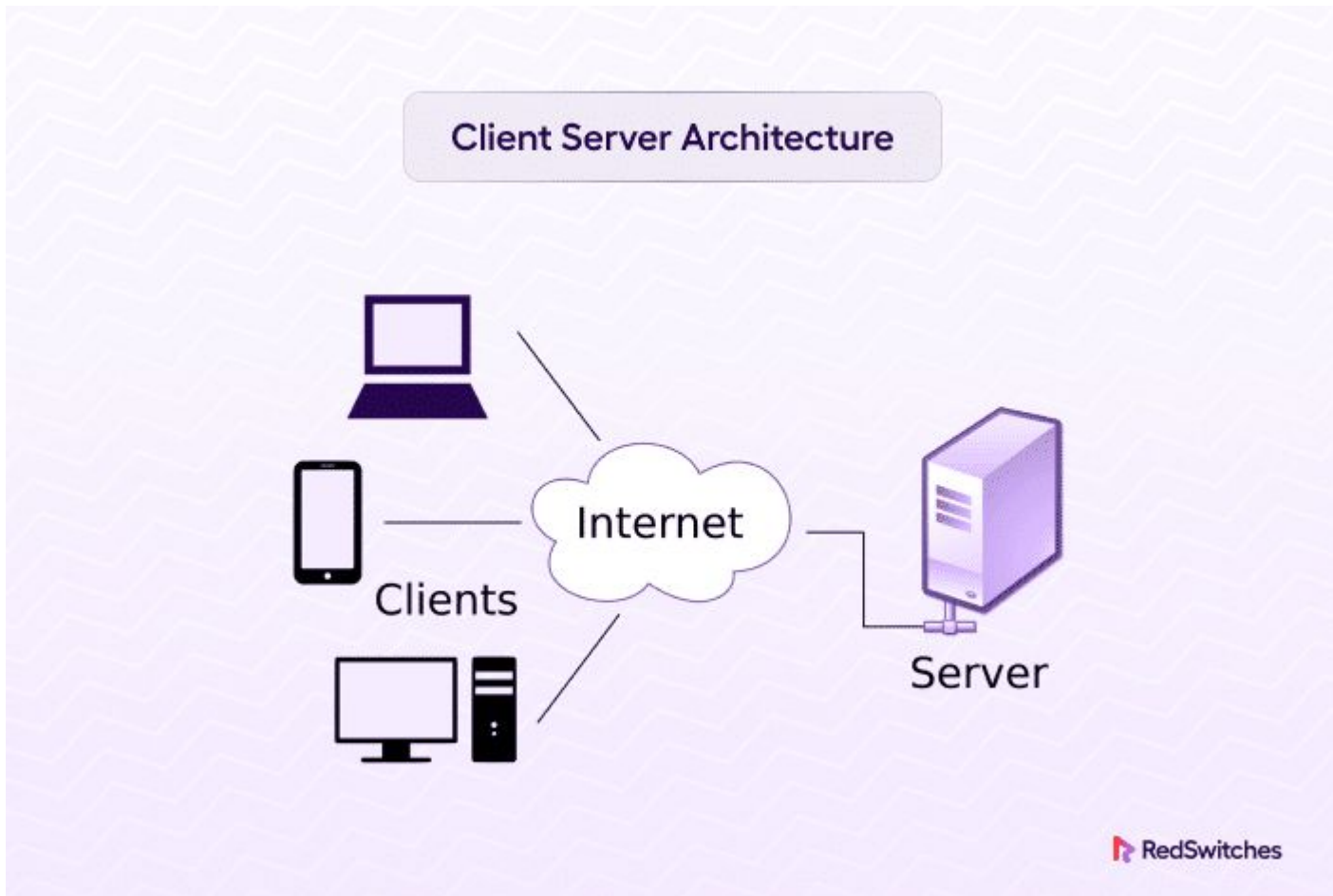
Centralized and Client Server architecture for the database



Centralized and Client Server architecture for the database

- A centralized database is basically a type of database that is stored, located as well as maintained at a single location only.
- This type of database is modified and managed from that location itself.
- The centralized location is accessed via an internet connection (LAN, WAN, etc).
- This centralized database is mainly used by institutions or organizations.

Centralized and **Client Server** architecture for the database



Centralized and Client Server architecture for the database

- The client-server architecture refers to a system that hosts, delivers, and manages most of the resources and services that the client requests.
- Client-server architecture, alternatively called a client-server model, breaks down tasks and workloads between clients and servers.
- The client sends a request for data, and the server accepts and accommodates the request, sending the data back to the user who needs them.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- **Entity Relationship Model: Introduction, Representation of entities, attributes, entity set**, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

E-R Model: Introduction, Representation of entities, attributes, entity set

- ❑ An entity is an **object in the real world** that is distinguishable from other objects.
- ❑ This is the **basic object of E-R Model**.
- ❑ Entities can be represented by '**Rectangle**'.

❑ Example:



- ❑ In a school database students, teachers, classes, and courses offered can be considered as entities.

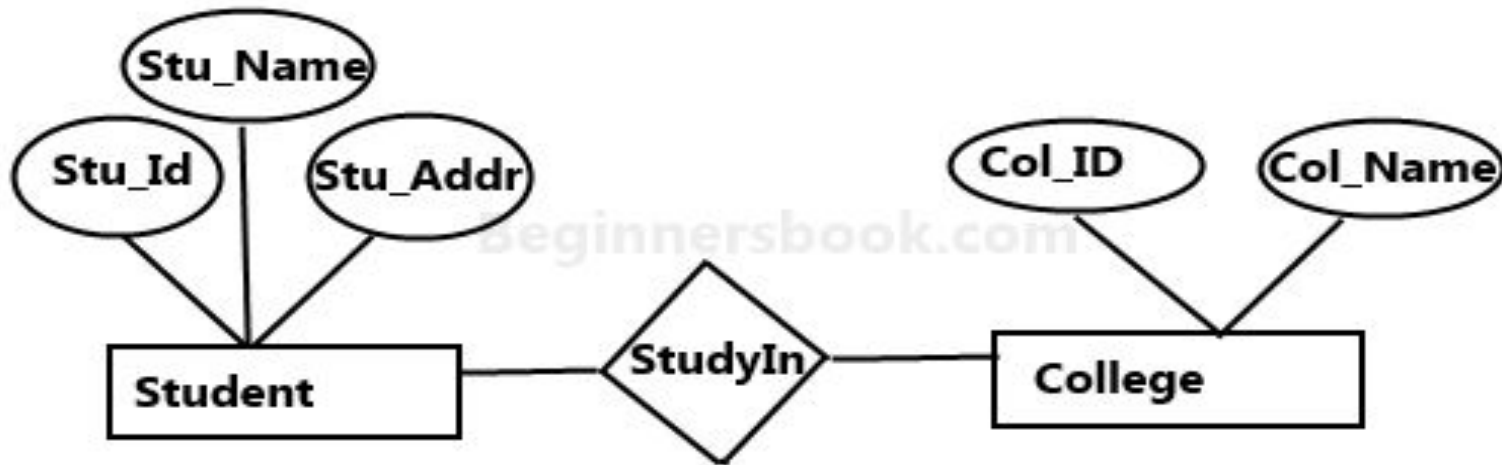
E-R Model: Introduction, Representation of entities, attributes, entity set

- Entities have *attributes*
 - **Example:** people have *names* and *addresses*
- An **entity set** is a set of entities of the same type that share the same properties.
 - **Example:** set of all persons, companies, trees, holidays

Roll No.	Name	Course	
CS08	Steive	Comp. Sci.	→ Entity
EE54	Jhoson	Electronics	→ Entity
B12	Eva	Biology	→ Entity
F32	Jhoson	Finance	→ Entity
M26	Erica	Maths	→ Entity

Student Table

E-R Model: Introduction, Representation of entities, attributes, entity set



Sample E-R Diagram

E-R Model: Introduction, Representation of entities, attributes, entity set

- ❑ Entities are represented by means of their properties, called **attributes**.
- ❑ For example, a student entity may have name, class, and age as attributes.
- ❑ **Domain** – the set of permitted values for each attribute.
- ❑ **For example,**
 - ❑ A student's name cannot be a numeric value.
 - ❑ A student's age cannot be negative, etc.

E-R Model: Introduction, Representation of entities, attributes, entity set

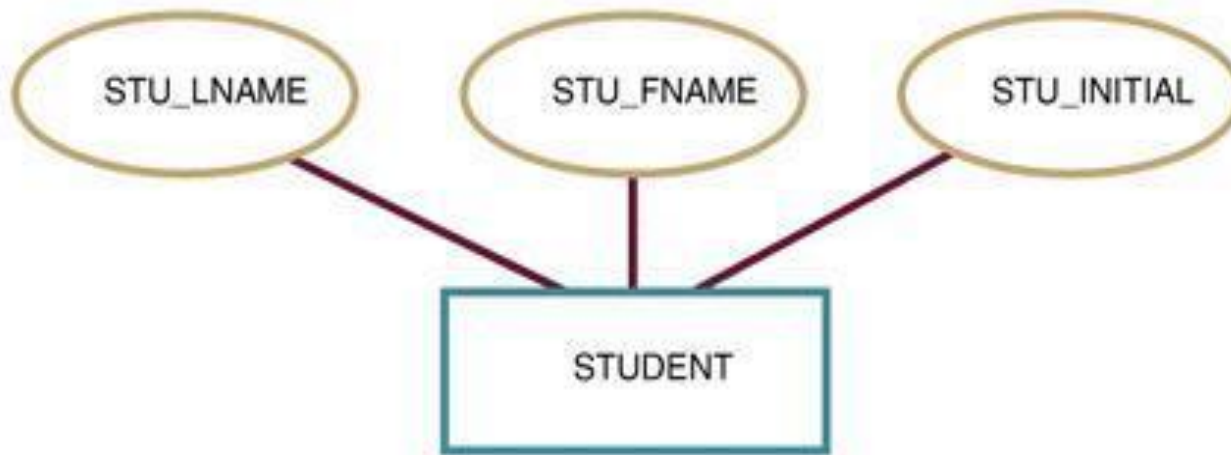


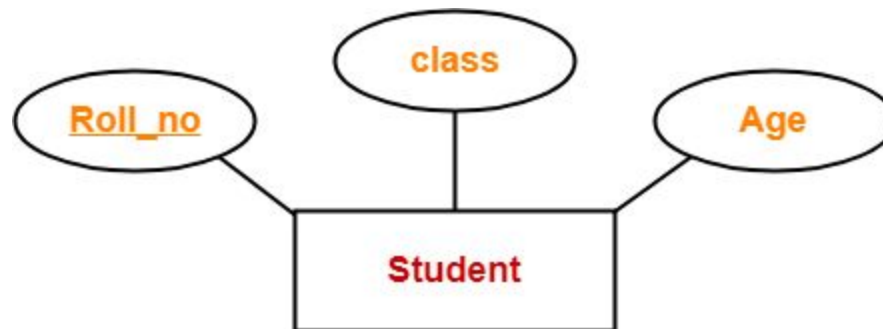
FIGURE 4.6 ■ THE ATTRIBUTES OF THE STUDENT ENTITY

E-R Model: Introduction, Representation of entities, attributes, entity set

- **Types of Attributes**
 - Simple attribute
 - Composite attribute
 - Derived attribute
 - Single-value attribute
 - Multi-value attribute
 - Key attribute

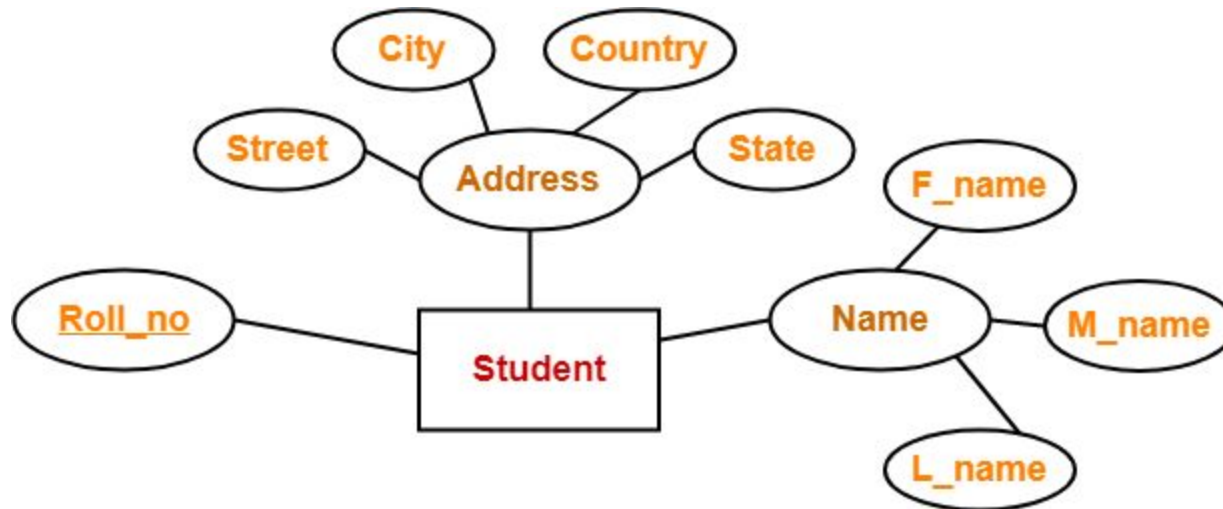
E-R Model: Introduction, Representation of entities, attributes, entity set

- Simple attribute / Atomic
- Simple attributes **are atomic values**, which cannot be divided further.
- **For example**, a student's phone number is an atomic value of 10 digits.



E-R Model: Introduction, Representation of entities, attributes, entity set

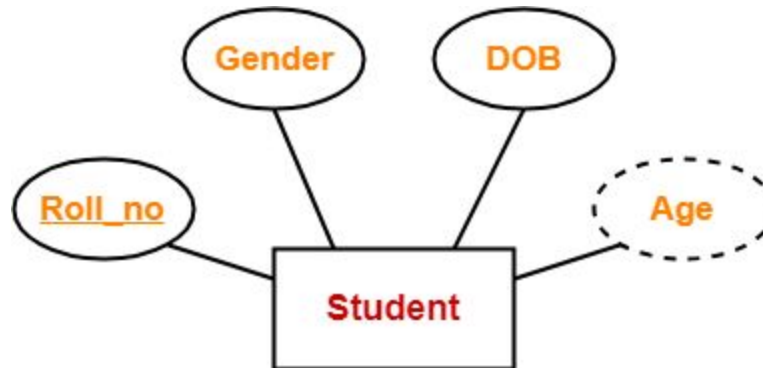
- Composite attribute / compound
- Composite attributes are made of more than one simple attribute.
- For example, a student's complete name may have first_name and last_name.



E-R Model: Introduction, Representation of entities, attributes, entity set

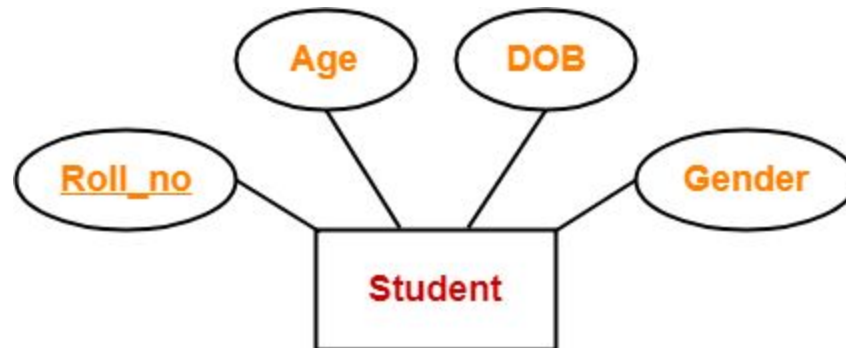
□ Derived attribute

- Derived attributes are the attributes that do not exist in the physical database, but their **values are derived from other attributes** present in the database.
- For another example, age can be derived from data_of_birth.



E-R Model: Introduction, Representation of entities, attributes, entity set

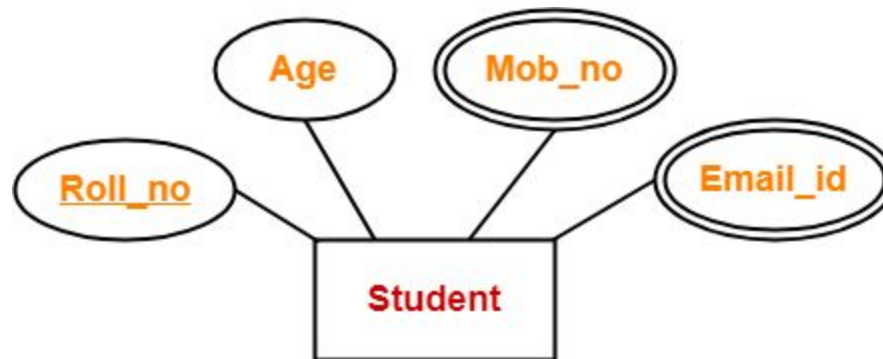
- Single-value attribute
- Single-value attributes contain single value.
- For example – Regd No, Aadhar....



E-R Model: Introduction, Representation of entities, attributes, entity set

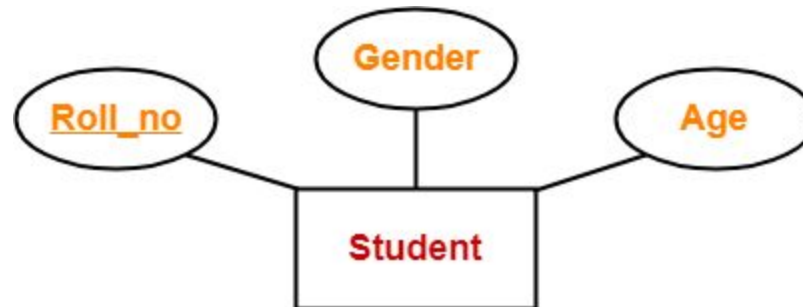
□ Multi-value attribute

- Multi-value attributes may contain more than one values.
- For example, a person can have more than one phone number, email_address, etc.



E-R Model: Introduction, Representation of entities, attributes, entity set

- Key Attribute :
- Represents **primary key**.
- It is an attribute, that has **distinct value for each entity**/element in an entity set.
- **For example**, Roll number in a Student Entity Type.



E-R Model: Introduction, Representation of entities, attributes, entity set

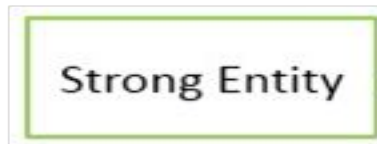
□ **Types of Entity:**

- Strong Entity
- Weak Entity

E-R Model: Introduction, Representation of entities, attributes, entity set

Strong Entity

- The **Strong Entity** is the one whose existence does not depend on the existence of any other entity in a schema.
- It is denoted by a **single rectangle**.
- A strong entity always has the **primary key** in the set of attributes that describes the strong entity.



E-R Model: Introduction, Representation of entities, attributes, entity set

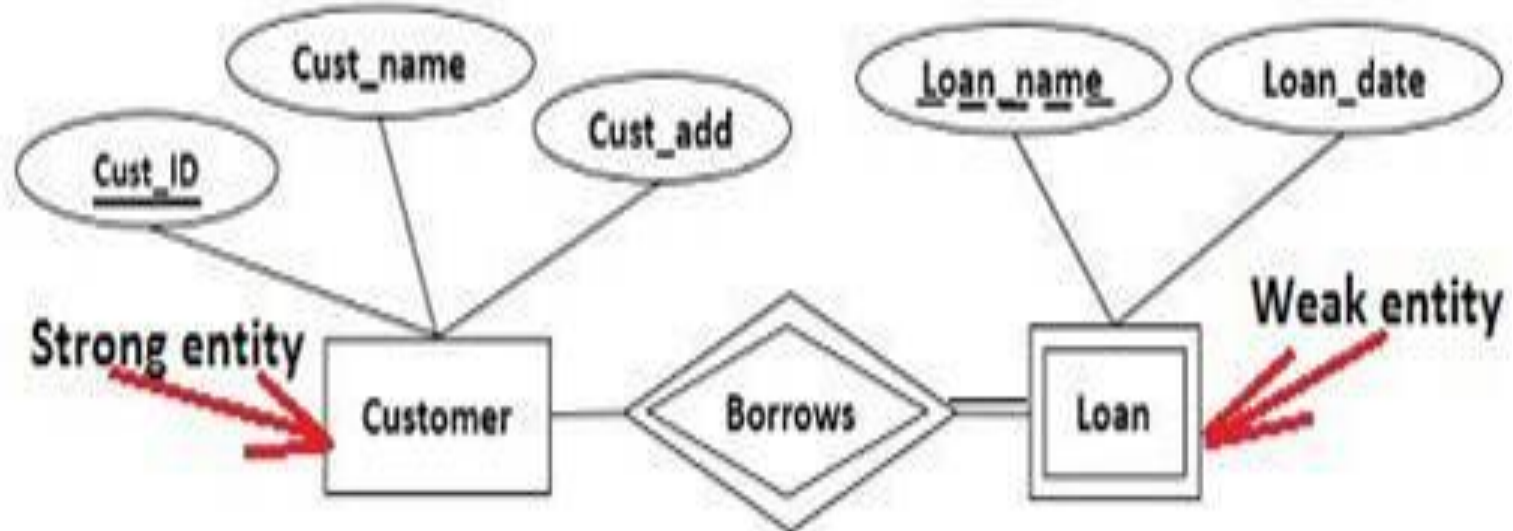
□ Weak Entity

- A **Weak entity** is the one that depends on its owner entity i.e. a strong entity for its existence.
- A weak entity is denoted by the **double rectangle**.
- Weak entity do **not** have the **primary key**.
- The relationship between a weak and a strong entity is denoted by Identifying Relationship **denoted with double diamond**.



E-R Model: Introduction, Representation of entities, attributes, entity set

□ Example

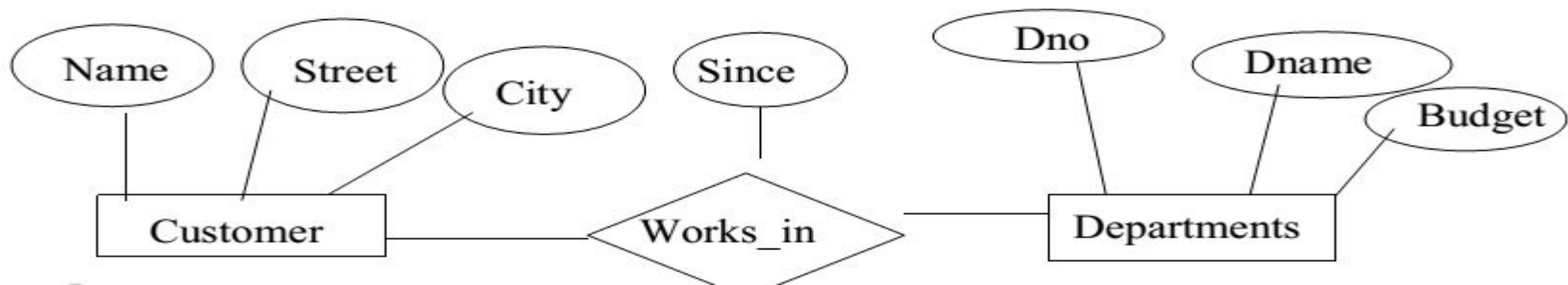


E-R Model: Introduction, Representation of entities, attributes, entity set

- **Types of Attributes**
 - Simple attribute
 - Composite attribute
 - Derived attribute
 - Single-value attribute
 - Multi-value attribute
 - Key attribute
 - * Descriptive Attribute

E-R Model: Introduction, Representation of entities, attributes, entity set

- **The attribute(s) used for describing the relationship** is called descriptive attributes, also referred as relationship attributes.
- They are actually used for storing information about the relationship.
- A relationship can have zero or more attributes.



UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, **relationship, relationship set**, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Relationship, Relationship set

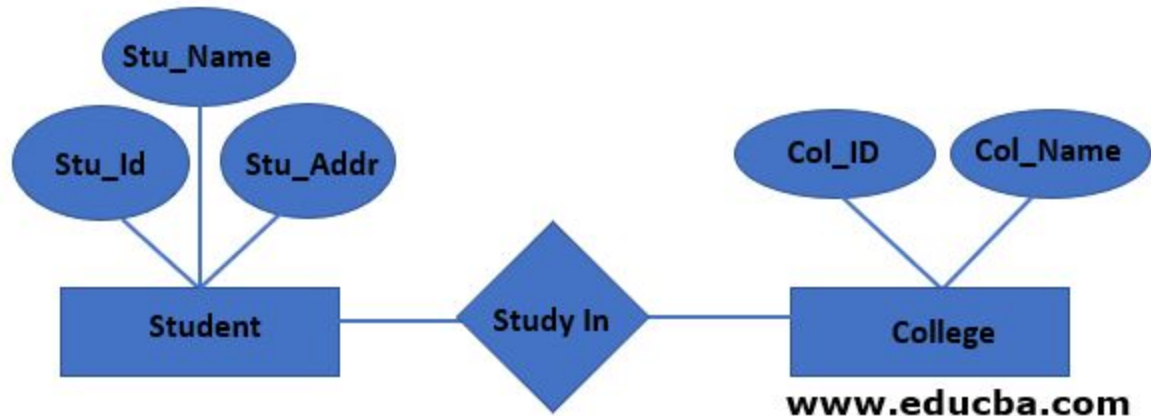
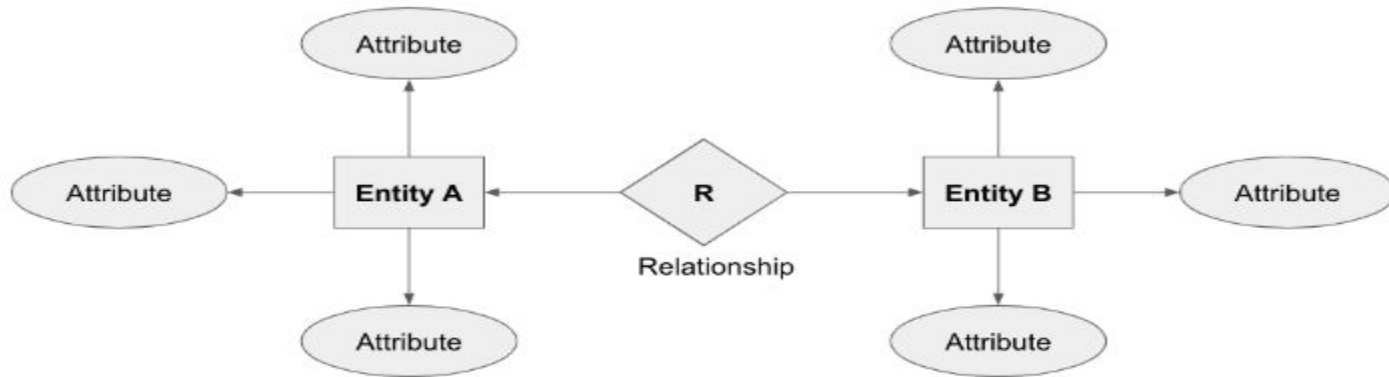


- Introduction
- Mapping Cardinalities / Types of Relationships

Relationship, Relationship set

- Introduction
- Mapping Cardinalities / Types of Relationships

Relationship, Relationship set



Relationship, Relationship set

- A relationship is an **association among two or more entities**.
- As with entities, we may wish to collect a **set of similar relationships into a relationship set**.
- A relationship can also have **descriptive attributes**.
- Descriptive attributes are used to **record information about the relationship**, rather than about any one of the participating entities.
- Intuitively, an **instance** can be thought of a '**snapshot**' of the relationship set at some instant in time.

Relationship, Relationship set

- The **number of roles** in the relationship:
 - Binary – links two entity sets;
 - Ternary – links three entity sets;
 - Nary – links n entity sets;
- **Degree of a Relationship Type**: The degree of a relationship type is the number of participating entity types.
- A relationship type of degree two is called **binary**, and one of degree three is called **ternary**.

Relationship, Relationship set

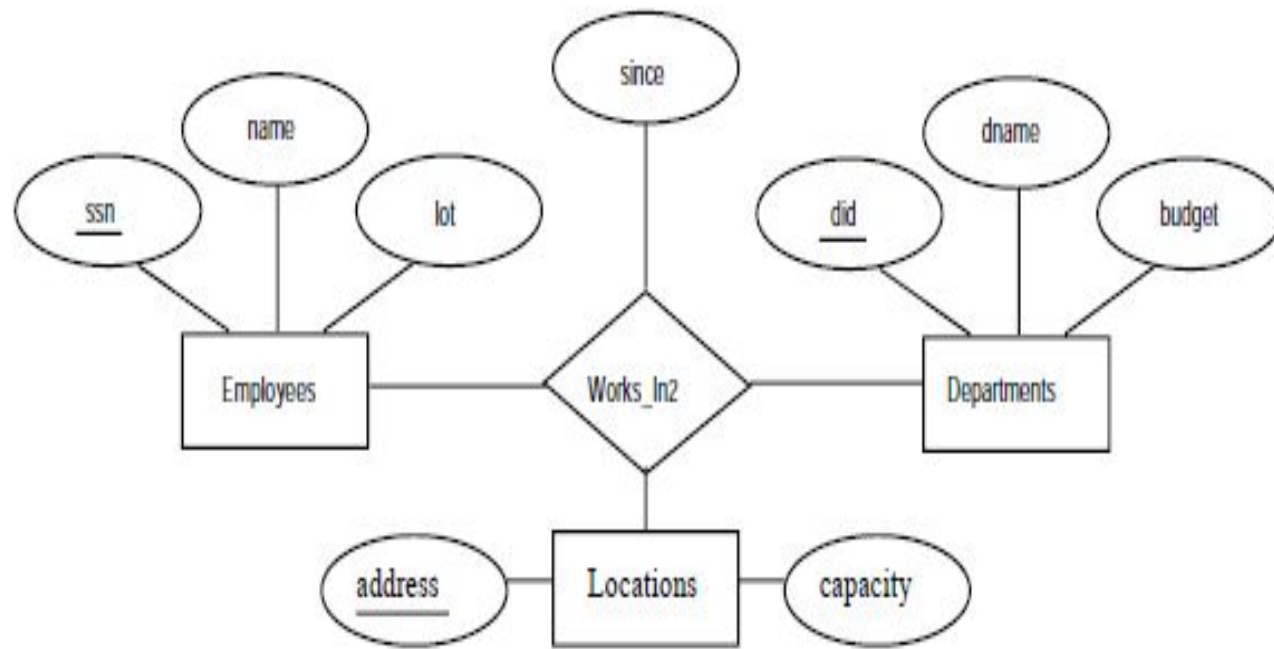


Figure 2.4 A Ternary Relationship Set

Relationship, Relationship set



- Introduction
- Mapping Cardinalities / Types of Relationships

Relationship, Relationship set

- One to One relationship
- One to many or many to one relationship
- Many to many relationships

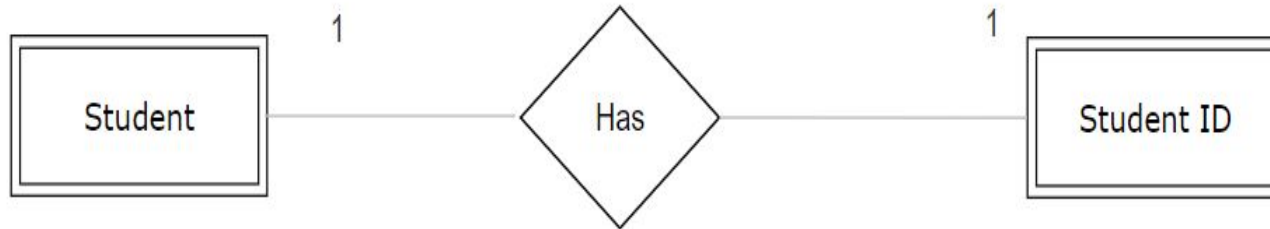
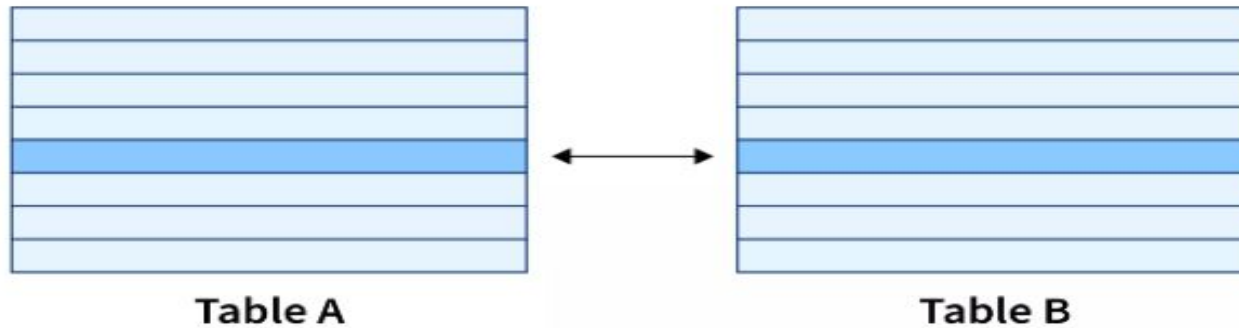
Relationship, Relationship set

- One to One relationship
- One to many or many to one relationship
- Many to many relationships

Relationship, Relationship set

- It is used to create a relationship between two tables in which a **single row of the first table can only be related to one and only one records of a second table.**
- Similarly, the **row of a second table can also be related to anyone row of the first table.**
- For example, If there are two entities, '**Person**'(Name, age, address, contact no.) and '**Aadhar card**'(Name, Aadhar no.).
- So each person can have only one Aadhar card, and the single Aadhar card belongs to only one person.
- You can visualize the one-to-one relationship in DBMS like this:

Relationship, Relationship set

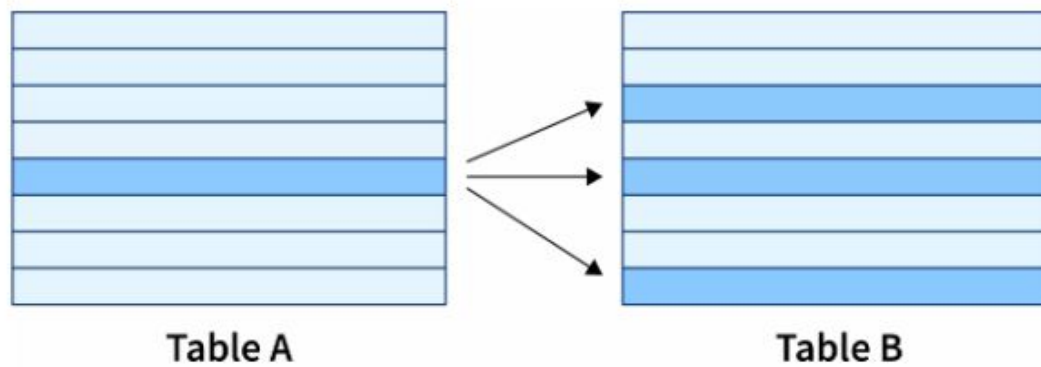


Relationship, Relationship set

- One to One relationship
- One to many or many to one relationship
- Many to many relationships

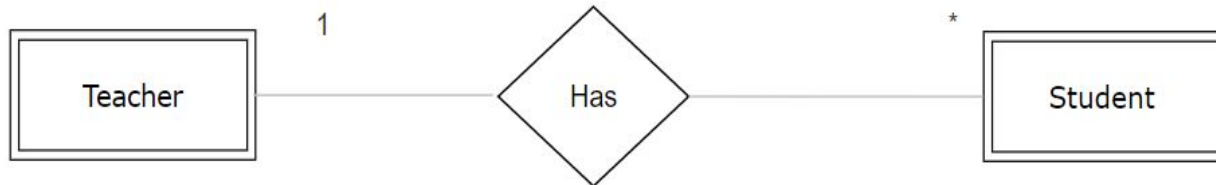
Relationship, Relationship set

- Any single rows of the first table can be related to one or more rows of the second tables, but the rows of second tables can only relate to the only row in the first table.
- It is also known as a **many to one** relationship.



Relationship, Relationship set

- Consider two relations: **Teacher** and **Student**. The relationship set between them can be characterized as “Teaches.”
- Each teacher teaches in a class, but a class can have numerous students. So one teacher can have many students.



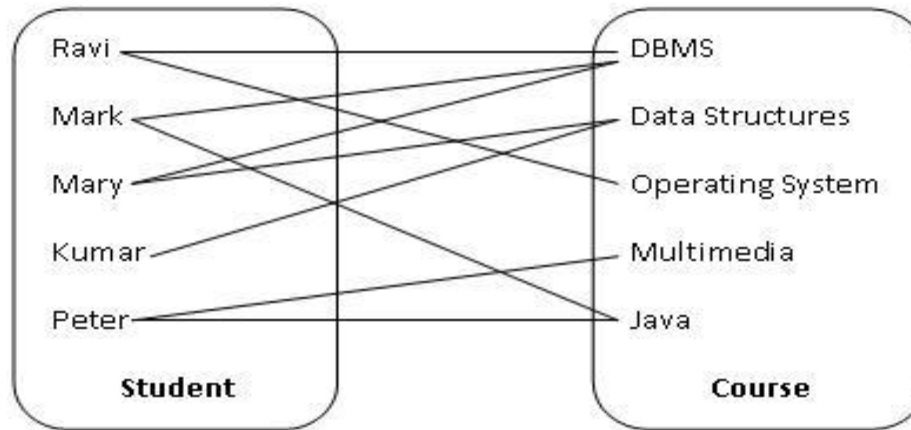
Relationship, Relationship set

- One to One relationship
- One to many or many to one relationship
- Many to many relationships

Relationship, Relationship set

- A many-to-many relationship arises when **multiple records in one entity can be linked to multiple records in another entity.**
- For example, consider the two tables i.e., a **student table and a courses table.**
- A particular student may enroll himself in one or more than one course, while a course also may have one or more students.
- A **doctor has many patients, and some patients visit many doctors.**

Relationship, Relationship set



UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, **constraints**, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Constraints

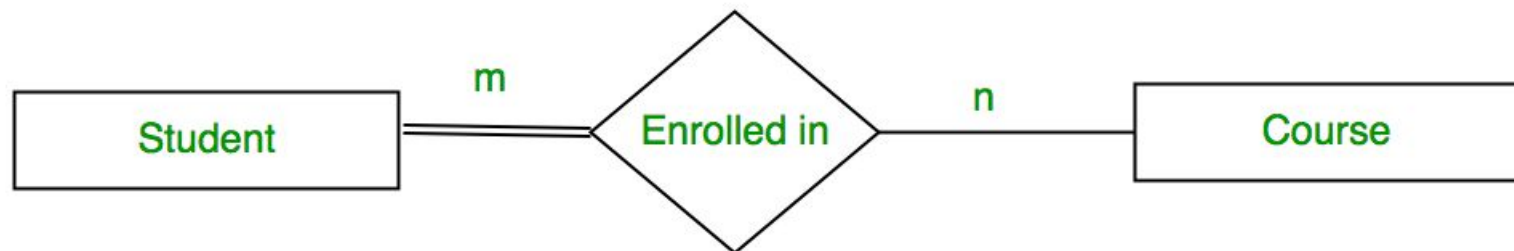
- Participation Constraints
 - Total/Mandatory Participation
 - Partial/Optional Participation

Constraints

- Participation Constraints
 - Total/Mandatory Participation
 - Partial/Optional Participation

Constraints

- Participation is said to be total if **every entity in E participates in at least one relationship in R**
- It is **indicated by a double line** (**==**) between entity and relationship.
- For example: If each student must enroll in a course, the participation of student will be total.

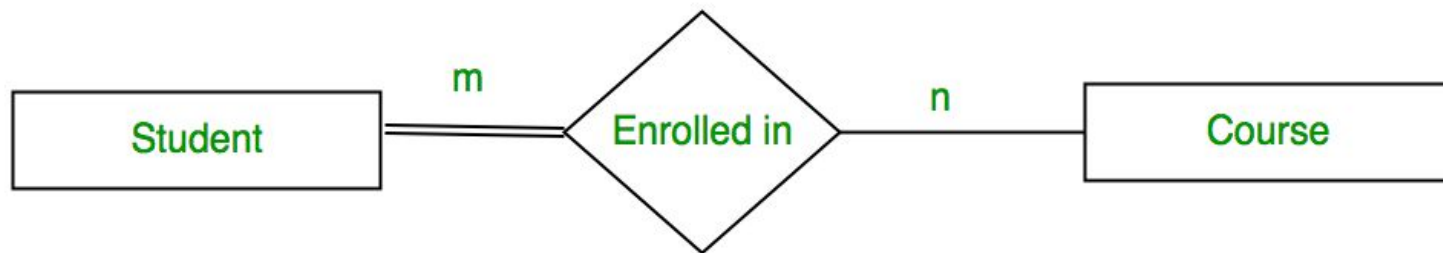


Constraints

- Participation Constraints
 - Total/Mandatory Participation
 - Partial/Optional Participation

Constraints

- Partial Participation – The entity in the entity set **may or may NOT participate in the relationship**.
- It is **indicated by a single line** (—) between entity and relationship.
- Example: If some courses are not enrolled by any of the student, the participation of course will be partial.



UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, **sub classes**, **super class**, inheritance, specialization, generalization using ER Diagrams.

Sub classes, Super class

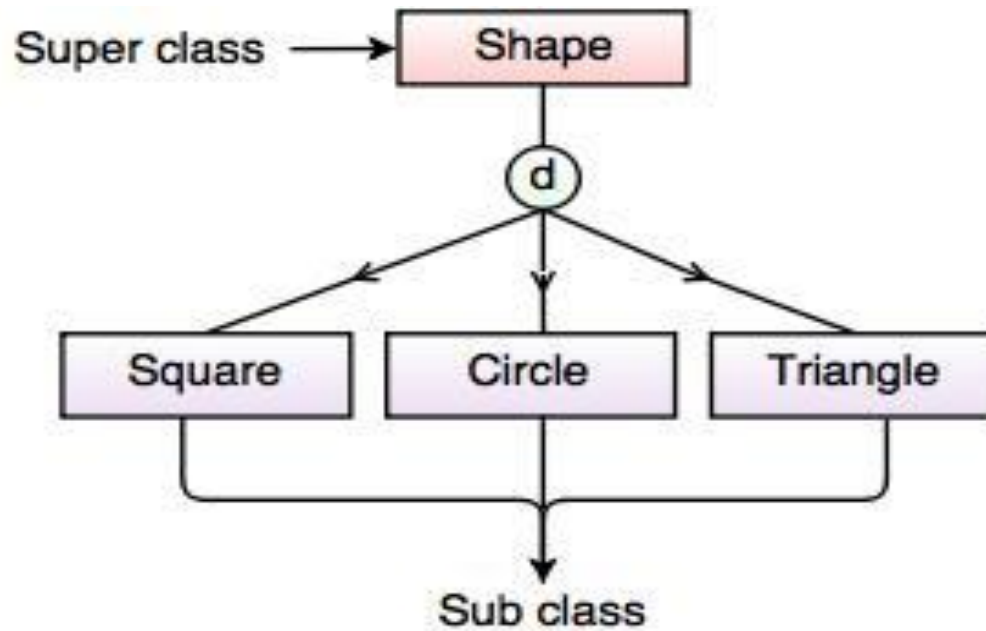
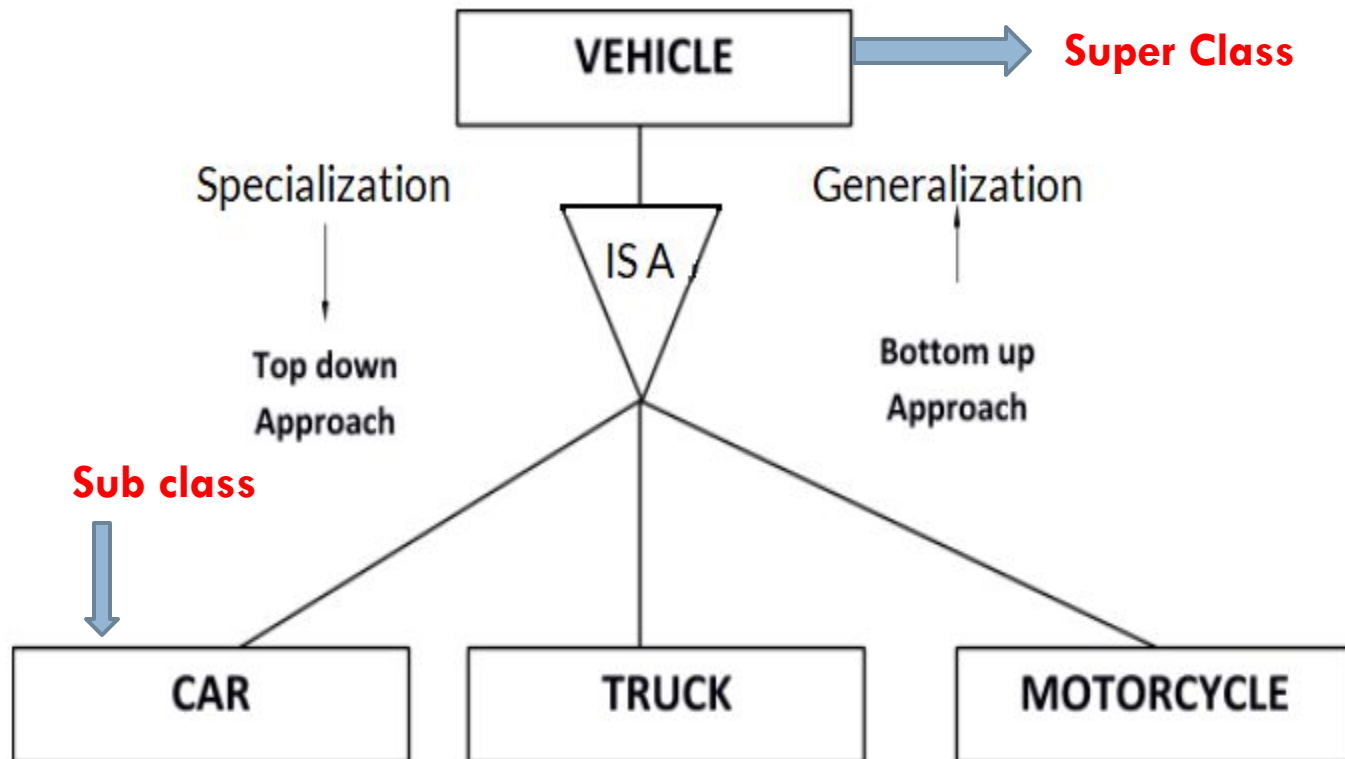


Fig. Super class/Sub class Relationship

Sub classes, Super class

- A **superclass** is the class from which many subclasses can be created.
- The **subclasses inherit the characteristics of a superclass.**
- The **superclass is also known as the parent class or base class.**
- **Specialized classes are often called subclass while a generalized class is called a superclass,** probably inspired by object-oriented programming.
- A sub-class is best understood by “**IS-A analysis**”.

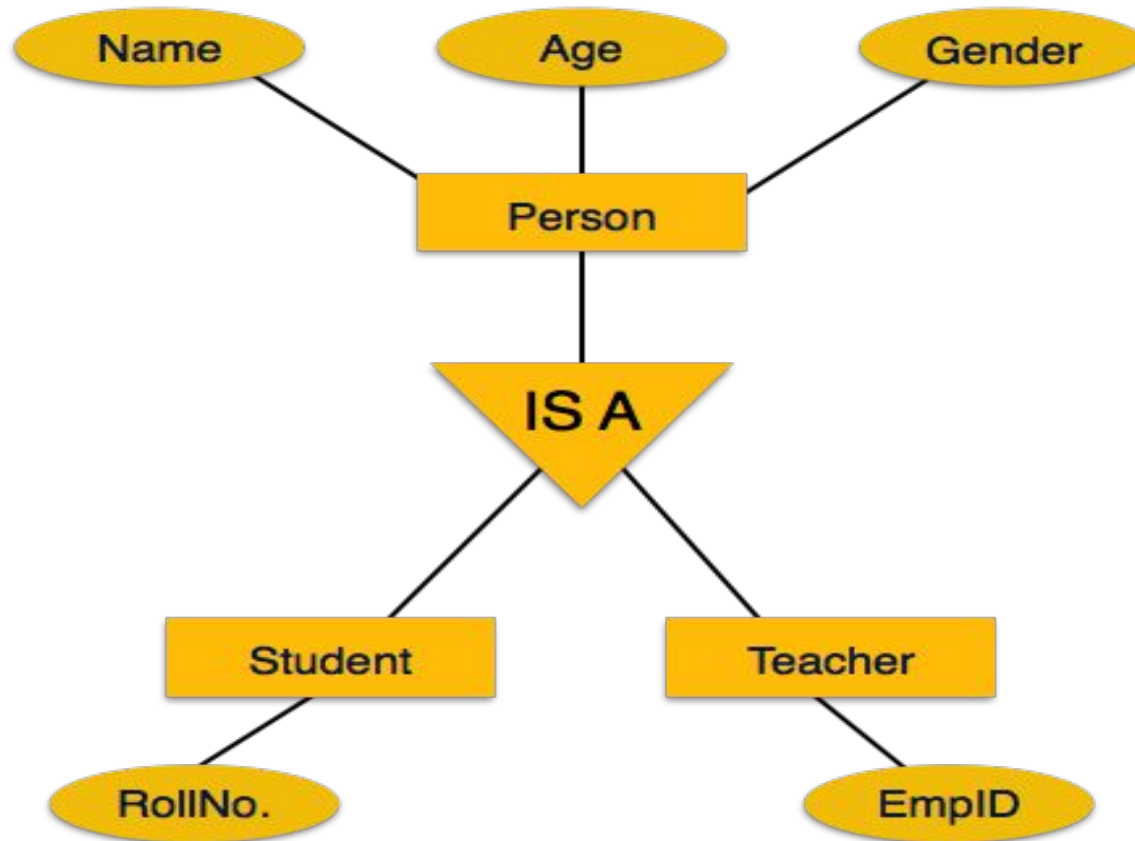
Sub classes, Super class



UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, **inheritance**, specialization, generalization using ER Diagrams.

Inheritance



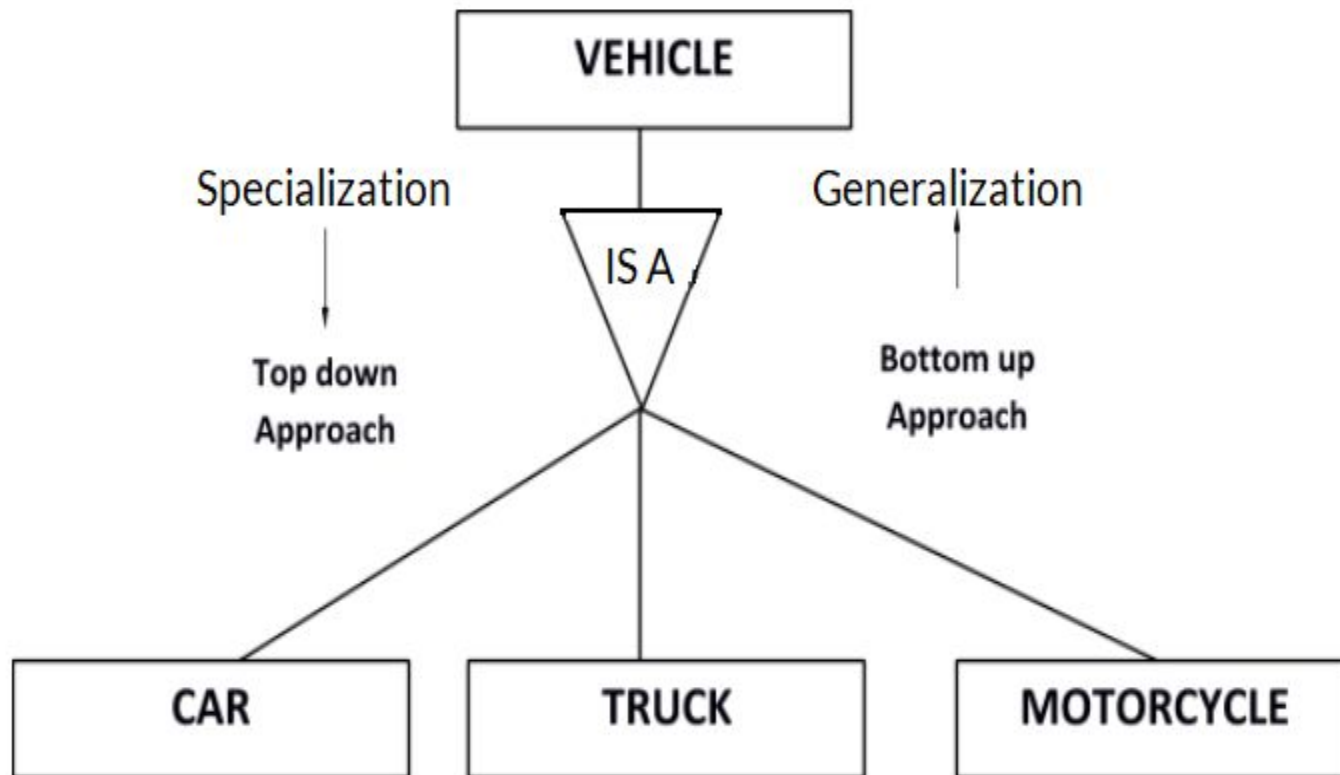
Inheritance

- Inheritance is basically the process of basing a class on another class i.e to build a class on a existing class.
- The new class contains all the features and functionalities of the old class in addition to its own.
- The class which is newly created is known as the subclass or child class and the original class is the parent class or the superclass.

UNIT I: Introduction

- Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database.
- Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, **specialization, generalization** using ER Diagrams.

Specialization, Generalization



Specialization, Generalization

- Inheritance is an important feature of generalization and specialization.
- In specialization, a **higher-level entity is divided into lower-level sub-entities** that inherit its attributes.
- In generalization, similar lower-level entities are combined into a higher-level entity that holds common attributes.
- In both cases, **inheritance allows sub-entities to reuse the properties of the parent entity.**

Specialization, Generalization

- Generalization is the process of extracting common properties from a set of entities and creating a generalized entity from it.
- It is a bottom-up approach in which two or more entities can be generalized to a higher-level entity if they have some attributes in common.

Specialization, Generalization

- In specialization, an entity is divided into sub-entities based on its characteristics.
- It is a top-down approach where the higher-level entity is specialized into two or more lower-level entities.